20XT87 - DATA MINING LAB

GENOMEFOLIO — DEMO GUIDE

Team members:

22PT17 - Mohamed Aashir S, 21PT10 - Kiruthika V G

Prerequisites

- Python 3.11 (managed with `uv`)
- No API keys. Prices are downloaded once through `yfinance` and cached.

Install

```
uv sync
```

Run the full pipeline

```
uv run genomefolio
```

This downloads the 20-stock universe on first use, takes the most recent 252 trading days, symbolizes and aligns the strings, clusters the stocks, and forecasts each cluster's composite over 21 trading days.

Run offline (committed fixture)

```
uv run genomefolio --use-fixture
```

The fixture contains 10 stocks over five years, so the demo works with no network access. A real run of this command produced:

```
Universe: 10 stocks | window=252 days | clusters=2 | silhouette=0.041

Cluster 0 (3 stocks) forecast=-0.0032 over 21 days [arima]
  JNJ, PG, KO

Cluster 1 (7 stocks) forecast=+0.0267 over 21 days [arima]
  AAPL, MSFT, INTC, CSCO, JPM, XOM, DIS
```

Read the output

- **Universe** — how many stocks are being grouped, the window length, the chosen number of clusters and its silhouette.
- **Cluster N** — the cluster id, how many stocks it contains, the forecast cumulative return over the horizon, and which model produced it (`arima`, `spline`, or `historical_mean` when the forecast fell back).

- The member list tells you which stocks form the cluster.

A silhouette close to zero (as above) is normal for daily return data: the clusters are a coarse grouping of weak structure, not a strong separation. Treat the forecast as a modest signal with a one-month horizon.

Save the result as JSON

```
uv run genomefolio --use-fixture --out forecasts.json
```

The file contains the cluster membership and the forecast for each cluster:


```
{
  "n_clusters": 2,
  "silhouette": 0.041,
  "forecast_horizon": 21,
  "clusters": { "0": ["JNJ", "PG", "KO"], "1": ["AAPL", "..."] },
  "forecasts": {
    "0": { "expected_return": -0.0032, "model": "arima", "order": [1, 0, 1], "fallback": false
  }
}
```


Try different settings

Edit `config/params.yaml` and re-run. Useful knobs:

- `window.train_days` — longer windows smooth out noise, shorter windows react faster.
- `symbolize.cutoffs` — move the flat band; `15/35/65/85` makes a wider flat region.
- `symbolize.quantile_scope` — `pooled` keeps relative volatility in the strings.
- `align.band` — how many days a pattern may shift.
- `cluster.min_cluster_size` and `k_cap_divisor` — how fine the grouping can become.
- `forecast.model` — `arima` or `spline`.
- `forecast.max_p` / `max_q` — size of the AIC grid.

Inspect stability across bands

The module also exposes a band sweep:

```
uv run python -c "from src.config import load_config; from src.data import load_fixture, to_re
```

If the cluster count changes with the band, the grouping is not robust.

Run the tests

```
uv run pytest
```

The suite covers symbolization edge cases, alignment against hand-worked examples, distance-matrix symmetry, clustering constraints, forecast fallbacks and the end-to-end pipeline.

Troubleshooting

- **"need at least 252 days of returns"** — the chosen universe or fixture has too little history; use a longer date range or the committed fixture.
- **Download errors** — `yfinance` needs network access; the cached CSV in `data/cache/` is reused after the first success.
- **ARIMA non-convergence** — handled: the forecast falls back to the historical mean and is flagged (`fallback: true`).